

Sprawozdanie z projektu

Ewolucja ekosystemów języków programowania na GitHubie (2011–2026)

Mateusz Lampert, Wojciech Maćkowiak

Projekt UMISI — Grupa 1 (projekty dziedzinowe i aplikacyjne)

Temat 5: Ewolucja ekosystemów języków programowania

Repozytorium: <https://github.com/McEnigma24/UMISI-sem1-Visualization>

10 czerwca 2026

Streszczenie

Projekt analizuje zmiany popularności 21 głównych języków programowania na podstawie aktywności typu `PushEvent` w publicznych repozytoriach GitHub w latach 2011–2026 (dane z GH Archive eksportowane przez BigQuery). Celem jest pokazanie trendów popularności w czasie, porównanie aktywności społeczności oraz wykrycie struktury podobieństwa ekosystemów językowych. Zbudowano kompletny potok przetwarzania danych w Pythonie (ETL → budowa wektorów cech → redukcja wymiaru → EDA → generowanie wizualizacji) oraz interaktywny dashboard (`index.html`) oparty na Vega-Lite i Plotly. Wektory profili czasowych poddano siedmiu metodom redukcji wymiaru (PCA, kernel PCA, t-SNE, UMAP, TriMAP, PaCMAP, OpenTSNE) w 2D i 3D oraz klastrowaniu metodą k -średnich. Sprawozdanie omawia źródło i preprocessing danych, architekturę rozwiązania, wyniki analizy oraz ich ostrożną interpretację.

Spis treści

1	Cel i zakres projektu	3
2	Dane i ich pozyskanie	3
2.1	Źródło danych	3
2.2	Preprocessing (ETL)	4
3	Architektura rozwiązania	4
3.1	Reprezentacja wielowymiarowa i transformacja cech	5
3.2	Warstwa wizualizacji (dashboard)	5
4	Wyniki analizy	5
4.1	Trendy aktywności w czasie	5
4.2	Mapa udziałów rynku	6
4.3	Przegląd kwartalny — treemap	6
4.4	Udziały języków w czasie	7
4.5	Aktywność społeczności	8

5	Analiza eksploracyjna (EDA)	9
5.1	Zmiana udziału i obserwacje nietypowe	9
5.2	Klastrowanie	9
6	Redukcja wymiaru	10
7	Interpretacja i wnioski	12
8	Ograniczenia	12
9	Odtworzenie wyników	12

1 Cel i zakres projektu

Projekt realizuje temat 5 z Grupy 1 zajęć UMISİ — *Ewolucja ekosystemów języków programowania*. Grupa 1 obejmuje projekty dziedzinowe i aplikacyjne oparte na dużych zbiorach danych, których wspólnym elementem jest przygotowanie reprezentacji wielowymiarowych, zastosowanie metod redukcji wymiaru, wizualizacja struktur w przestrzeni 2D oraz — często — klastrowanie i wykrywanie obserwacji nietypowych.

Cel projektu: pokazanie zmian popularności języków programowania i technologii w czasie na podstawie danych o aktywności w repozytoriach GitHub.

Zakres:

- przygotowanie danych o aktywności repozytoriów per język,
- analiza popularności języków w czasie,
- porównanie aktywności społeczności kontrybutorów,
- interaktywne wykresy trendów,
- analiza zmian dominujących technologii i koncentracji rynku,
- element redukcji wymiaru: budowa wektorów profili czasowych i porównanie wyników wielu metod (m.in. UMAP, TriMAP, PaCMAP),
- interpretacja wyników.

Analizą objęto **21 najpopularniejszych języków** (wg aktywności `PushEvent`): JavaScript, Python, Java, TypeScript, C#, C++, PHP, C, Go, Ruby, Rust, Kotlin, Swift, Dart, Scala, R, Objective-C, Lua, Haskell, Julia, Perl. Horyzont czasowy obejmuje okres od początku GH Archive (2011) do ostatniego kwartału w danych (**2026-04-01**, tj. Q2 2026).

2 Dane i ich pozyskanie

2.1 Źródło danych

Dane pochodzą z **dwóch publicznych zbiorów dostępnych w Google BigQuery**, które zostały **złączone po repozytorium** (zapytanie `sql/bigquery_export.sql`):

1. **GH Archive** (`githubarchive.day.20*`) — strumień *zdarzeń* GitHub. Z niego pobieramy **aktywność**: zdarzenia `PushEvent` (`commit`) oraz `actor.id` (kto pushował) wraz z identyfikatorem repozytorium, do którego trafił push. To jest źródło „ile się działo” — liczba commitów i unikalnych aktorów w czasie.
2. **GitHub Repos** (`bigquery-public-data.github_repos.languages`) — snapshot *zawartości* repozytoriów. Z niego pobieramy **skład językowy** każdego repo: dla każdego języka liczbę bajtów kodu (`lang.name`, `lang.bytes`). To jest źródło „z czego repo jest zbudowane” — na jakie języki dany commit „wędruje”.

Idea złączenia. Sam strumień zdarzeń nie mówi, w jakim języku napisano commit — mówi tylko, do którego repozytorium trafił. Dlatego dla każdego repozytorium liczymy z drugiego zbioru *wagi językowe* (`lang_fraction = bytes(język)/suma_bytes(repo)`), a następnie każdemu `PushEvent` przypisujemy języki repozytorium proporcjonalnie do tych wag. Złączenie odbywa się po znormalizowanym „slugu” repozytorium (`właściciel/nazwa`). W efekcie jeden push do repo zbudowanego np. w 55% z C++ i 45% z C dokłada 0,55 do licznika C++ i 0,45 do licznika C.

Eksportowany plik CSV (wynik zapytania) ma następujący schemat:

Kolumna	Znaczenie
month / quarter	oś czasu (miesiąc YYYY-MM albo pierwszy dzień kwartału YYYY-MM-DD)
language	nazwa języka ze snapshotu <code>github_repos.languages</code>
push_events	suma wag <code>lang_fraction</code> po wszystkich pushach w okresie („push-równoważne” ułamki; wartość niecałkowita)
unique_actors	liczba unikalnych <code>actor.id</code> (z GH Archive)

Charakter metryki. Kolumna `push_events` to *ważona* aktywność (suma ułamków bajtowych), a nie „goła liczba `PushEvent`”. Dzięki temu jest spójna z udziałami procentowymi i koncentracją rynku, ale jej *absolutna skala* zależy od definicji wag oraz od kompletności obu zbiorów — w szczególności od tego, dla ilu repozytoriów istnieje snapshot składu językowego (repo bez wpisu w `github_repos.languages` nie wnosi wag).

2.2 Preprocessing (ETL)

Potok `src/etl.py`:

1. filtrowanie do 21 wybranych języków,
2. obliczenie udziałów w każdym okresie (`share_pct`),
3. metryki społeczności: unikalni aktorzy, intensywność `events_per_actor`, ranking,
4. koncentracja rynku: indeks HHI oraz udział trzech liderów (top 3).

Luka czasowa 2013–2014. Surowy eksport BigQuery (`manyLanguages.csv`) bywa niespójny — np. skok z 2012 do 2015. Aby wykresy nie miały dziury czasowej, brakujące kwartały uzupełniono **interpolacją liniową** skryptem `src/interpolate_quarter_gaps.py`; domyślnym wejściem ETL jest plik wynikowy `data/raw/real/manyLanguages_added.csv`.

Pliki pośrednie (`data/processed/`) obejmują m.in. `monthly_activity.csv`, `monthly_shares.csv`, `community_metrics.csv`, `market_concentration.csv`, `language_vectors.csv`, `dim_reduction.csv` / `dim_reduction_3d.csv`, `language_profiles.csv`, `language_clusters.csv` oraz `pca_explained_variance`.

3 Architektura rozwiązania

Projekt nie przechowuje gotowych wykresów jako jedynej źródła prawdy — **wizualizacje są generowane skryptami** z plików CSV. Uruchomienie `python src/run_pipeline.py` wykonuje kolejne kroki potoku i składa dashboard `index.html`.

Krok	Skrypt	Zadanie
1	<code>etl.py</code>	wczytanie i filtrowanie danych, udziały, metryki, koncentracja
2	<code>build_vectors.py</code>	długi format wektorów cech (profil czasowy języka)
3	<code>dim_reduction.py</code>	7 metod redukcji wymiaru w 2D i 3D
4	<code>eda.py</code>	statystyki per język, klastry <i>k</i> -means, wariancja PCA, profile
5	<code>build_viz.py</code>	główny generator: Altair → Vega-Lite, Plotly 3D, montaż <code>index.html</code>
6	<code>build_yearly_viewer.py</code>	strona rocznego podglądu (treemap / siatka)
7	<code>build_market_snapshot_viewer.py</code>	strona mapy udziałów (kwartalna)
8	<code>build_concentration_viewer.py</code>	strona koncentracji rynku

3.1 Reprezentacja wielowymiarowa i transformacja cech

Z tabeli udziałów budowana jest macierz profili: **wiersze = języki, kolumny = kolejne okresy** (pivot `share_pct`, braki $\rightarrow 0$). Na macierz nakładana jest transformacja `share_pct` $\rightarrow$ `log1p` $\rightarrow$ `StandardScaler` (standaryzacja *per kolumna*, czyli per okres czasu). Wynik to macierz o wymiarach $21 \times T$, gdzie T to liczba unikalnych okresów (w bieżącym eksporcie $T = 62$ kwartały). Każdy wiersz to jeden język jako punkt w przestrzeni $\mathbb{R}^T$ — to wejście do redukcji wymiaru, PCA i klastrowania.

3.2 Warstwa wizualizacji (dashboard)

Dashboard `index.html` łączy:

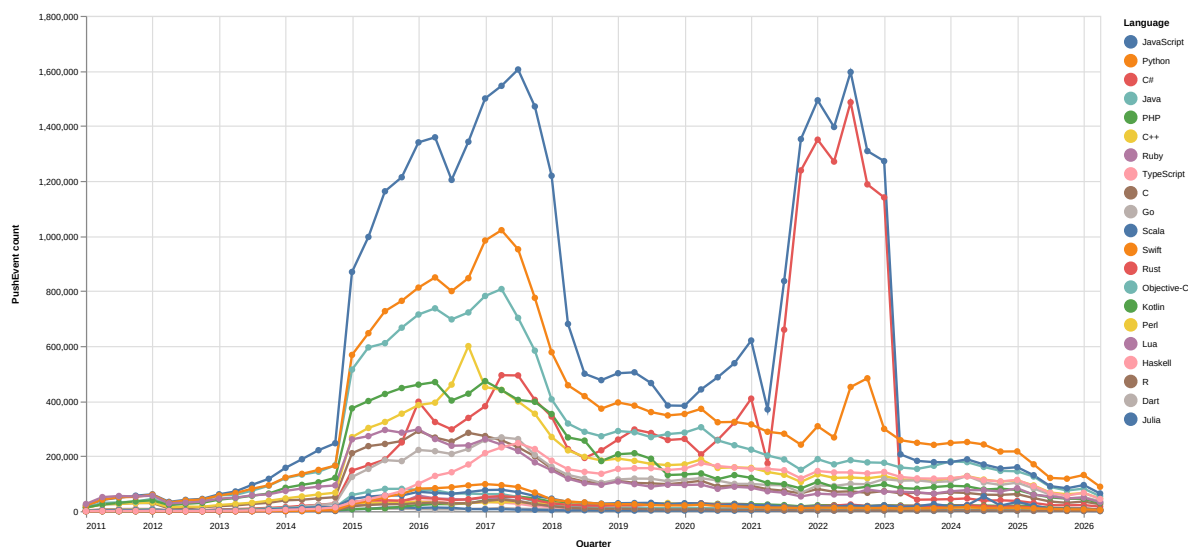
- **wykresy Vega-Lite** (`*.vl.json`) renderowane przez `vega-embed` — interaktywne (tool-tipy, zoom);
- **scenę 3D Plotly** osadzoną bezpośrednio w stronie (działa również przy otwarciu z dysku, `file:///`);
- **trzy strony-„viewery”** (roczny treemap, mapa udziałów, koncentracja) ładowane w `iframe` z animacją w czasie.

Zakładki dashboardu: trendy aktywności, udziały w czasie, mapa udziałów, przegląd kwartalny treemap, koncentracja rynku, porównanie społeczności, EDA (zmiana udziału, wariancja PCA, klastrowanie) oraz redukcja wymiaru 2D i 3D.

4 Wyniki analizy

4.1 Trendy aktywności w czasie

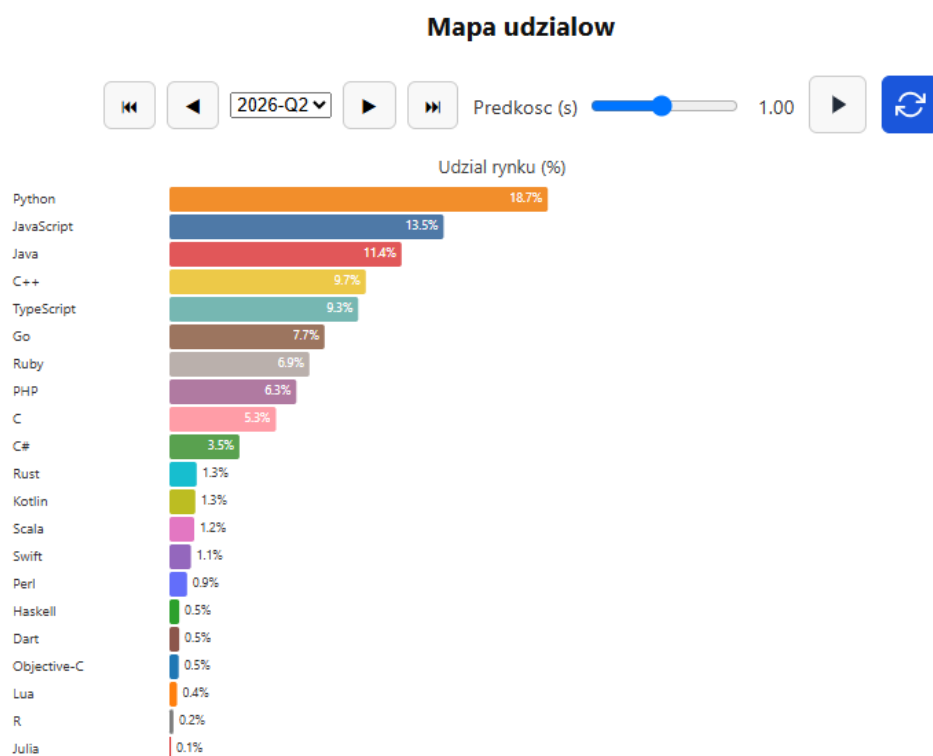
Wykres trendów (rys. 1) pokazuje sumę kwartalną ważonych „push-równoważnych” zdarzeń per język. Widoczny jest ogólny wzrost aktywności po ok. 2015, szczyt w latach **2021–2023** oraz spadek po 2023. JavaScript i Python utrzymują wysokie poziomy, TypeScript wyraźnie rośnie.



Rysunek 1: Trendy aktywności: liczba (ważonych) zdarzeń `PushEvent` per język w funkcji kwartału (2011–2026).

4.2 Mapa udziałów rynku

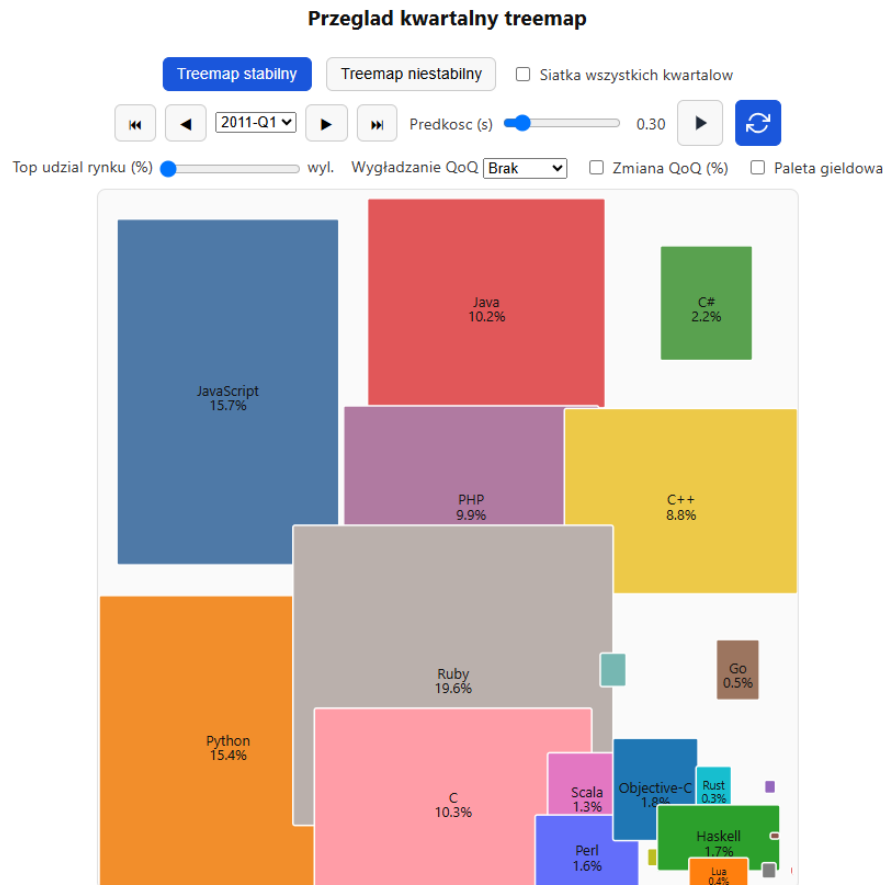
Mapa udziałów (rys. 2) przedstawia procentowy udział języków w aktywności w ostatnim dostępnym kwartale (Q2 2026). Dominują Python (18,7%), JavaScript (13,5%) oraz Java (11,4%); dalej C++, TypeScript i Go. Widać charakterystyczny „długi ogon” języków niszowych (R, Julia, Lua) o udziałach poniżej 1%.



Rysunek 2: Mapa udziałów rynku (%) w kwartale Q2 2026 — interaktywny widok z osią czasu (animacja kwartał po kwartale).

4.3 Przegląd kwartalny — treemap

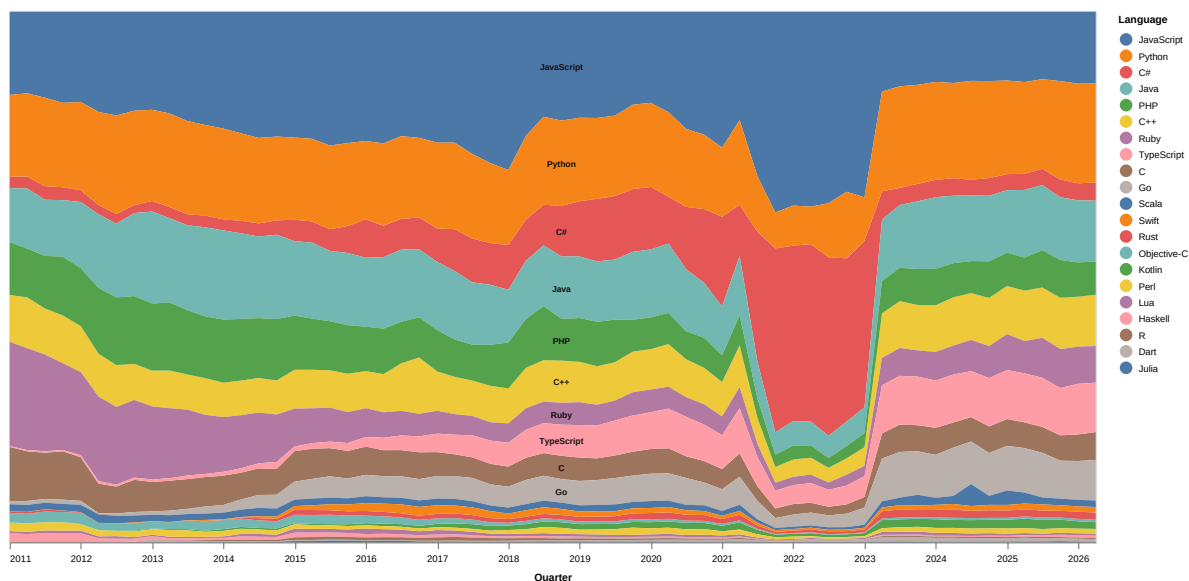
Treemap (rys. 3) reprezentuje wielkość ekosystemu jako pole prostokąta (analogia „mapy giełdy”). Widok jest animowany w czasie; zrzut pokazuje wczesny okres (Q1 2011), gdy strukturę rynku tworzyły jeszcze JavaScript, Java, Ruby, PHP, C i Python — przed późniejszą ekspansją TypeScriptu, Go czy Rusta.



Rysunek 3: Przegląd kwartalny (treemap) struktury udziałów — stan dla Q1 2011.

4.4 Udziały języków w czasie

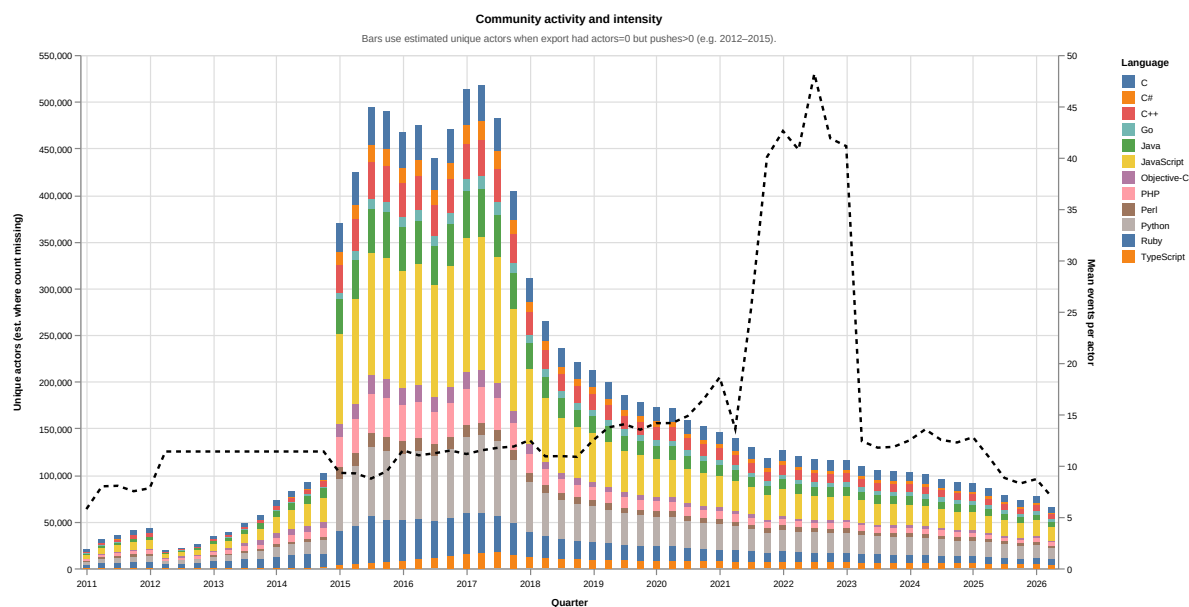
Rysunek 4 przedstawia skumulowany wykres obszarowy (*stacked area*) udziałów poszczególnych języków w całkowitej aktywności w kolejnych kwartałach. W przeciwieństwie do trendów absolutnych ten widok eksponuje *względna* popularność: dobrze widać stopniowe „kurczenie się” udziału JavaScriptu, ekspansję Pythona i TypeScriptu oraz erozję udziału Ruby i PHP, mimo że bezwzględne wolumeny rosną dla większości ekosystemów.



Rysunek 4: Udziały języków w aktywności w czasie (skumulowany wykres obszarowy, 2011–2026).

4.5 Aktywność społeczności

Porównanie społeczności. Wykres (rys. 5) łączy dwa kanały: słupki = suma `unique_actors` dla top 12 języków (szerokość bazy kontrybutorów), czarna linia = średnia `events_per_actor` (intensywność). Wzrost linii przy stabilnych słupkach sugeruje wzrost intensywności (więcej zdarzeń na aktora), a nie samej liczby osób. Należy pamiętać, że ta sama osoba kontrybuująca w kilku językach jest liczona wielokrotnie — suma słupków *nie* jest globalnym „headcountem”.

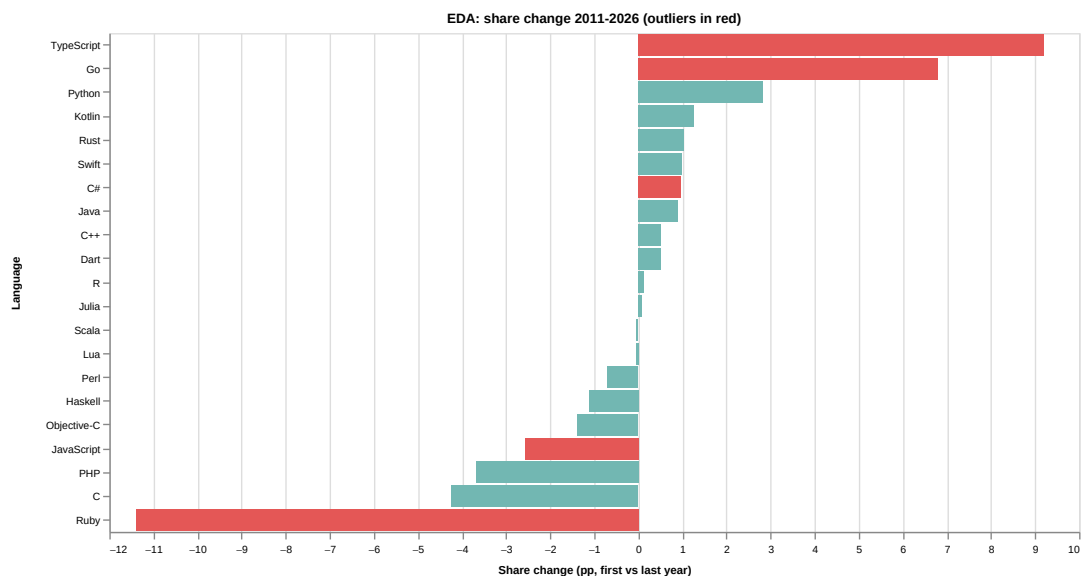


Rysunek 5: Porównanie społeczności: szerokość bazy aktorów (słupki) vs. średnia intensywność zdarzeń na aktora (linia).

5 Analiza eksploracyjna (EDA)

5.1 Zmiana udziału i obserwacje nietypowe

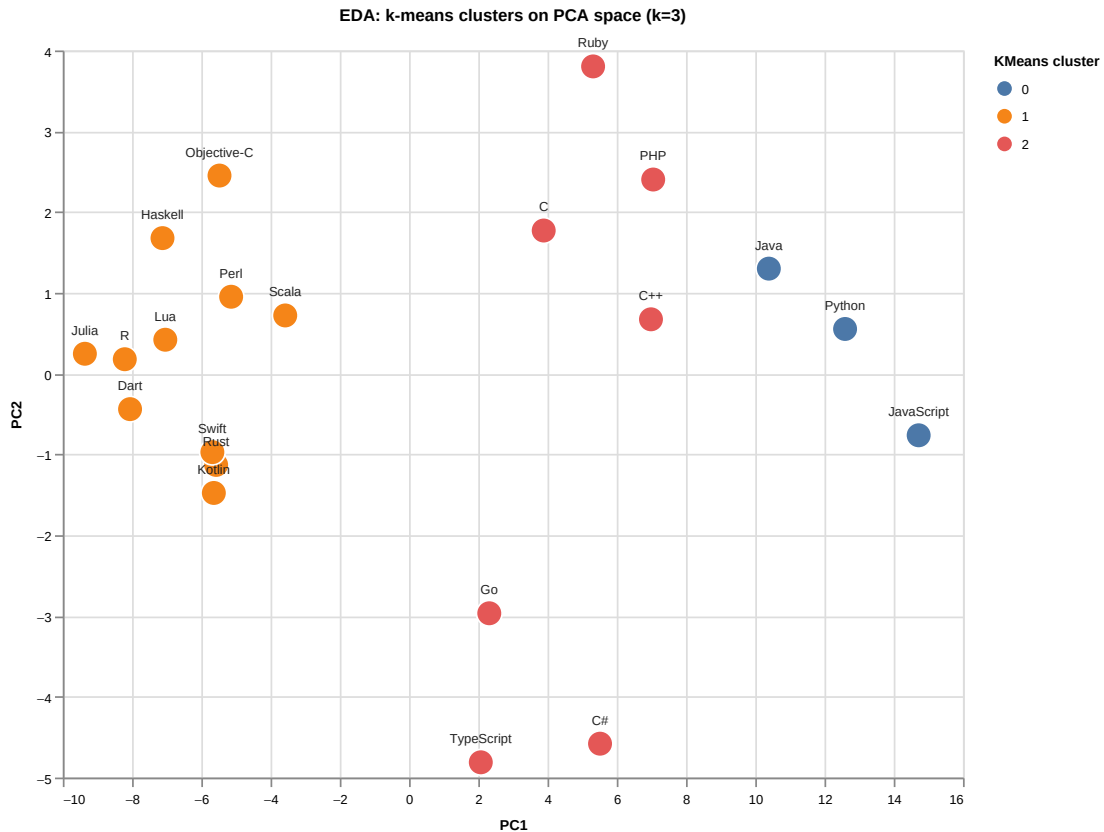
Dla każdego języka policzono zmianę udziału (pierwszy vs. ostatni rok, w punktach procentowych) oraz nachylenie trendu (regresja liniowa). Obserwacje nietypowe (outliery) wykrywano z z -score zmienności i zmiany udziału, próg $|z| > 1,5$. Rysunek 6 pokazuje, że największe *wzrosty* dotyczą TypeScriptu, Go i Pythona, a największe *spadki* — m.in. starszych ekosystemów; Ruby i PHP tracą udział, podczas gdy Rust rośnie z niskiej bazy (typowy profil „emerging language”).



Rysunek 6: Zmiana udziału w aktywności (pp, pierwszy vs. ostatni rok). Wartości dodatnie = wzrost popularności.

5.2 Klastrowanie

Na znormalizowanych wektorach profili czasowych ($21 \times T$) zastosowano *k-means* ($k = 3$, `random_state=42`). Punkty na rys. 7 są umieszczone w przestrzeni dwóch pierwszych składowych PCA, a kolor oznacza przypisanie do klastra. Wyłaniają się trzy grupy: (i) duże, dominujące ekosystemy (JavaScript, Python, Java), (ii) szeroka grupa języków średnich i niszowych oraz (iii) ekosystemy o specyficznym profilu zmienności (m.in. Ruby, PHP, C, C++).



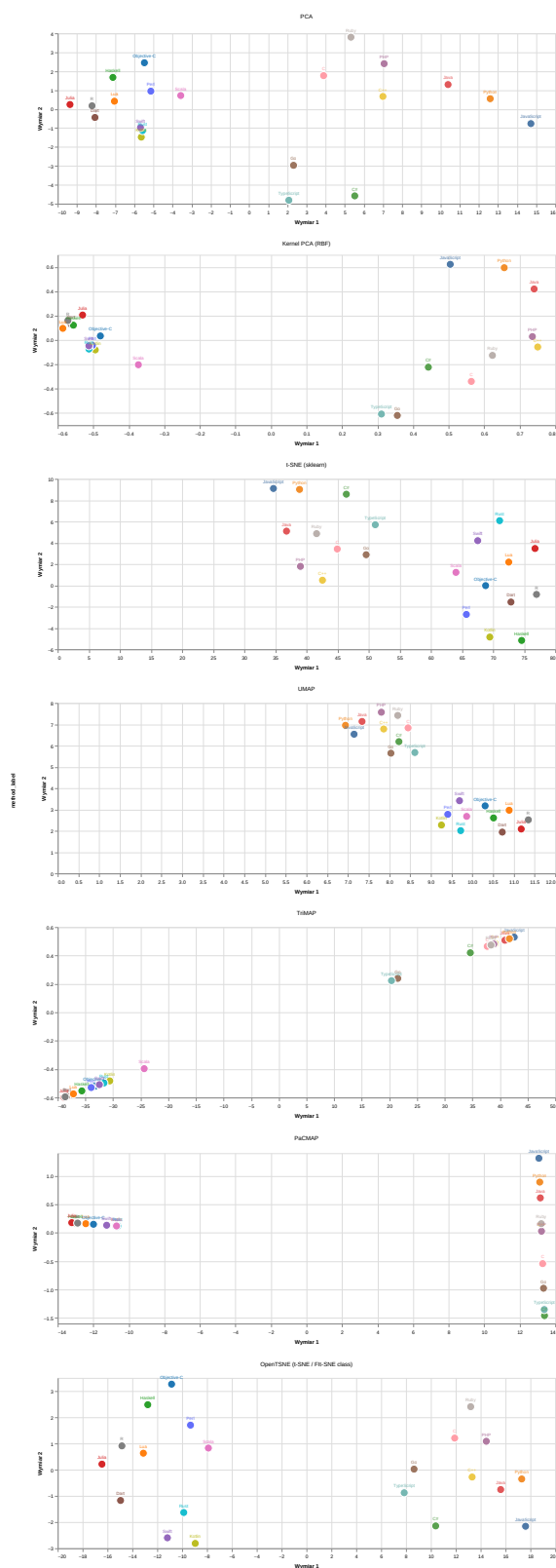
Rysunek 7: Klastrowanie k -means ($k = 3$) na profilach czasowych, rzutowane na płaszczyznę PC1–PC2.

6 Redukcja wymiaru

Wektory profili czasowych (każdy język = punkt w $\mathbb{R}^T$, $T = 62$) zrzutowano na 2D siedmioma metodami omawianymi na zajęciach. Każda metoda kładzie nacisk na inną własność struktury danych:

Metoda	Charakterystyka
PCA	liniowa baza — dominujące kierunki zmian w czasie
Kernel PCA (RBF)	nieliniowe odwzorowanie w przestrzeń cech, potem 2D
t-SNE (sklearn)	klasyczny t-SNE (Barnes–Hut) — lokalne sąsiedztwa
UMAP	zachowanie struktury lokalnej i części globalnej
TriMAP	nacisk na zachowanie tripletów odległości
PaCMAP	balans bliskich i dalekich par — czytelna separacja klastrów
OpenTSNE	szybki t-SNE (optymalizacja bliska FIt-SNE)

Rysunek 8 zestawia wszystkie siedem paneli. Panele mają **niezależne osie** (`resolve_scale`), więc nie porównujemy liczbowo współrzędnych między metodami — istotne są *względne sąsiedztwa* w obrębie jednego panelu. We wszystkich metodach języki rosnące (TypeScript, Rust, Python) wyraźnie odróżniają się od spadających (Ruby, PHP), a ekosystemy enterprise (Java, C#) lokują się pośrodku. Dodatkowo dashboard udostępnia interaktywną **scenę 3D** (Plotly) z tymi samymi metodami.



Rysunek 8: Redukcja wymiaru — 2D rzuty profili językowych siedmioma metodami (PCA, kernel PCA, t-SNE, UMAP, TriMAP, PaCMAP, OpenTSNE). Etykiety na punktach; osie niezależne między panelami.

7 Interpretacja i wnioski

- **JavaScript** pozostaje liderem aktywności, ale jego *udział* stopniowo maleje wraz z różnicowaniem ekosystemu.
- **Python** i **TypeScript** wyraźnie zyskują — Python dzięki data/ML i DevOps, TypeScript dzięki adopcji we frontendzie i full-stacku.
- **Rust** rośnie z niskiej bazy; **Ruby** i **PHP** tracą udział.
- Rynek pozostaje **skoncentrowany** — kilka języków dominuje (wysoki HHI, duży udział top 3).

Ostrożna interpretacja skoków. Wzrost „wszystkiego” po ok. 2015 częściowo odzwierciedla realną dynamikę (rosnąca baza publicznego kodu), a częściowo *kontekst danych* (niespójność starszej części GH Archive, łączenie eksportów, interpolacja luk). Szczyt **2021–2023** *nie pokrywa się* z pierwszym lockdownem 2020 — COVID jest tylko jednym z elementów tła. Bardziej wiarygodna jest narracja **wieloczynnikowa**: opóźniona reakcja gospodarki, druga fala cyfryzacji (remote, cloud), fale hype’u (web3/NFT, boom ML/AI), produkty GitHub (np. Copilot od 2021) oraz aktywność botów/CI. Spadek po 2023 może wynikać z normalizacji po przegraniu, redukcji zatrudnienia w tech oraz przesunięcia pracy do repozytoriów prywatnych (których archiwum *nie* widzi). Zaleca się rozdzielać **trend względny** (udziały, rankingi) od **trendu absolutnego** (ważone pushy) i zawsze przypominać definicję metryki.

8 Ograniczenia

- Dane = **publiczny** GitHub, nie cały rynek oprogramowania; aktywność prywatna jest niewidoczna.
- Skład językowy pochodzi ze snapshotu `github_repos.languages` — jest statyczny w czasie (te same wagi dla repo w całej historii) i może być nieaktualny lub niekompletny.
- Absolutna skala „ważonych pushy” zależy od definicji wag i kompletności archiwum.
- Zakres czasu domyślnego pliku kończy się na 2026-04-01; lata 2025+ dołączane są osobnym zapytaniem.
- Część starszego okresu (2012–2015) zawiera niespójności i interpolowane kwartały.

9 Odtworzenie wyników

```
pip install -r requirements.txt
python src/run_pipeline.py
# następnie otwórz index.html w przeglądarce
```

Domyślnym wejściem ETL jest `data/raw/real/manyLanguages_added.csv` (eksport BigQuery); inny plik: `python src/etl.py --input ścieżka.csv`. Wyniki metod stochastycznych (klastrowanie *k*-means, t-SNE, UMAP itd.) są powtarzalne dzięki ustalonemu ziarnu `RANDOM_SEED = 42` w `src/config.py`.

Zgodność z wymaganiami Grupy 1. Dashboard interaktywny (`index.html`), EDA i wniośki (`src/eda.py`), opis źródła i preprocessingu (`data/README.md`, `sql/`, `src/etl.py`), reprezentacja wielowymiarowa (wektory profili), redukcja wymiaru wieloma metodami z zajęć, klastro-

wanie (k -means) oraz wykrywanie obserwacji nietypowych — wszystkie wymagania tematu 5 zostały zrealizowane.